Federated Fine-Tuning of Large Language Models via LoRA Adapters: A Flower-Based Simulation Study

Krish Patel
Department of Computer Engineering and Computer Science
California State University, Long Beach

Ayush Alpesh Gupta
Department of Computer Engineering and Computer Science
California State University, Long Beach

***Abstract*—Fine-tuning large language models (LLMs) in a federated learning (FL) setting poses significant challenges related to communication overhead, data heterogeneity, and computational cost. This paper presents a simulation study of federated fine-tuning using Low-Rank Adaptation (LoRA) adapters, implemented with the Flower federated learning framework and applied to the Microsoft Phi-2 (2.7B parameter) model on the Alpaca-GPT4 instruction-following dataset. We employ 4-bit quantization (QLoRA) to reduce per-client memory requirements and transmit only LoRA adapter weights (approximately 11.8M trainable parameters, or 0.42% of the full model) during aggregation. We conduct four experiments comparing IID and non-IID data partitions, varying client participation fractions and communication rounds. Our results demonstrate that federated LoRA fine-tuning achieves consistent training loss reduction across all configurations, with the non-IID setting exhibiting a 5.5× larger fairness gap compared to IID baselines. Increasing client participation from 50% to 75% reduces this gap by 27%, while extending training to 10 rounds yields the highest token accuracy of 2.49%. Communication cost scales linearly at approximately 90 MB per round. These findings provide practical insights for deploying parameter-efficient federated fine-tuning of LLMs under realistic data heterogeneity conditions.**

***Index Terms*—Federated Learning, Large Language Models, LoRA, Parameter-Efficient Fine-Tuning, Flower Framework, QLoRA, FedAvg**

I. Introduction

Large language models (LLMs) have demonstrated remarkable capabilities across a wide range of natural language processing tasks, from instruction following to code generation and complex reasoning [1]. However, their deployment in privacy-sensitive domains such as healthcare, finance, and legal services presents a fundamental tension: achieving strong task performance requires fine-tuning on domain-specific data, yet this data often cannot be centralized due to regulatory constraints (e.g., HIPAA, GDPR) and institutional data-sharing policies [2].

Federated learning (FL) offers a principled framework for collaborative model training without sharing raw data [3]. In FL, multiple clients train on their local data and communicate only model updates to a central server, which aggregates them to produce an improved global model. The canonical Federated Averaging (FedAvg) algorithm [3] has been successfully applied to a variety of model architectures, but its direct application to LLMs with billions of parameters faces two critical challenges: (i) the communication cost of transmitting full model weights is prohibitive, and (ii) the memory footprint of fine-tuning the entire model exceeds the capacity of most client devices.

Parameter-Efficient Fine-Tuning (PEFT) methods, and Low-Rank Adaptation (LoRA) [4] in particular, address both challenges simultaneously. LoRA freezes the pretrained model weights and introduces small trainable rank-decomposition matrices into each transformer layer. This reduces the number of trainable parameters by orders of magnitude (typically to less than 1% of the full model), proportionally reducing both communication overhead and memory requirements. When combined with quantization techniques such as QLoRA [5], the memory footprint is further reduced, enabling fine-tuning of multi-billion parameter models on consumer-grade GPUs.

In this paper, we present a simulation study of federated LoRA fine-tuning using the Flower FL framework [6] applied to the Microsoft Phi-2 model (2.7B parameters) [7]. We use the Alpaca-GPT4 dataset [8] for instruction-following fine-tuning and investigate the effects of data heterogeneity, client participation fraction, and number of communication rounds on convergence, fairness across clients, and communication efficiency. Our key contributions are:

1) A complete, reproducible implementation of federated QLoRA fine-tuning using Flower's simulation API, with only LoRA adapter weights exchanged during aggregation.
2) A systematic comparison of four experimental configurations examining the interplay between data distribution (IID vs. non-IID), client sampling rate, and communication rounds.
3) Quantitative analysis of convergence behavior, client fairness (measured by validation loss gap), communication cost, and token-level prediction accuracy across all settings.

II. Related Work

A. Federated Learning

McMahan et al. [3] introduced the FedAvg algorithm, which remains the most widely used aggregation strategy in federated learning. Subsequent work has addressed key challenges including data heterogeneity [9], communication efficiency [10],

and system heterogeneity [11]. The non-IID data problem, where client data distributions differ significantly, has been shown to degrade convergence and produce unfair outcomes across clients [9].

B. Parameter-Efficient Fine-Tuning

LoRA [4] introduces low-rank decomposition matrices $\mathbf{A}$ and $\mathbf{B}$ into the attention weight matrices of transformer models, such that the weight update is represented as $\mathbf{W}' = \mathbf{W} + \mathbf{BA}$, where $\mathbf{B} \in \mathbb{R}^{d \times r}$ and $\mathbf{A} \in \mathbb{R}^{r \times d}$ with rank $r \ll d$. This reduces the trainable parameter count from $\mathcal{O}(d^2)$ to $\mathcal{O}(2dr)$. QLoRA [5] extends this approach with 4-bit NormalFloat quantization of the base model weights, double quantization for memory efficiency, and paged optimizers for handling memory spikes.

C. Federated PEFT for LLMs

Recent work has explored combining federated learning with parameter-efficient methods for LLMs. FedPara [12] and FedIT [13] demonstrated that transmitting only adapter weights can reduce communication costs by 99% or more while maintaining competitive performance. Zhang et al. [14] studied the interaction between non-IID data distributions and LoRA rank selection in federated settings. Our work complements these studies by providing a Flower-based implementation with systematic ablation across multiple experimental dimensions.

III. Methodology

A. System Architecture

Our system follows the standard federated learning architecture with a central server and $N = 8$ simulated clients. The Flower framework [6] manages client orchestration, parameter serialization, and aggregation. Each communication round consists of three phases:

1) The server broadcasts the current global LoRA adapter weights to a selected subset of clients.
2) Each selected client performs local fine-tuning on its private data partition.
3) The server aggregates the updated adapter weights using FedAvg with sample-count-proportional weighting.

Algorithm 1 summarizes the federated LoRA fine-tuning procedure.

Algorithm 1 Federated LoRA Fine-Tuning (FedLoRA)

Require: Base model $\mathcal{M}$, LoRA config (r, α), N clients, K clients/round, T rounds
Initialize global LoRA parameters θ^0
for $t = 1$ to T **do**
  $S_t \leftarrow$ sample K clients from $\{1, \ldots, N\}$
  for each client $k \in S_t$ **in parallel do**
    $\theta_k^t \leftarrow \theta^{t-1}$ {Download global LoRA weights}
    $\theta_k^t \leftarrow \textsc{LocalTrain}(\mathcal{M}, \theta_k^t, \mathcal{D}_k)$
    Upload θ_k^t to server
  end for
  $\theta^t \leftarrow \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j} \theta_k^t$ {FedAvg}
end for
return θ^T

B. Model and LoRA Configuration

We use Microsoft Phi-2 [7] as the base LLM, a 2.7-billion parameter decoder-only transformer. The model is loaded with 4-bit NormalFloat quantization via the `bitsandbytes` library, reducing the base model memory footprint from approximately 10.8 GB (FP32) to roughly 1.7 GB. LoRA adapters are applied to the query (`q_proj`), key (`k_proj`), value (`v_proj`), and output (`dense`) projection layers of each attention block with the following configuration:

- Rank: $r = 8$
- Scaling factor: $\alpha = 16$
- Dropout rate: 0.05
- Bias: none

This yields 11,796,480 trainable parameters, constituting 0.42% of the total 2,791,480,320 model parameters.

C. Dataset and Partitioning

We use the Alpaca-GPT4 dataset [8], an instruction-following dataset comprising 52,000 examples of instruction-input-output triplets generated by GPT-4. We subsample 800 training and 200 validation examples for computational feasibility. Each example is formatted into a structured prompt with `Instruction`, optional `Input`, and `Response` sections.

For data partitioning, we implement two strategies:

IID Partitioning. Examples are randomly shuffled and distributed uniformly across 8 clients in round-robin fashion, yielding approximately 100 training examples per client.

Non-IID Partitioning. Examples are first categorized into semantic groups—short responses, long responses, and code-related tasks—based on response length and content keywords. Clients are assigned primary group preferences, and examples are distributed to produce intentional imbalance in both sample count (ranging from 20 to 261 per client) and content distribution. This mimics realistic scenarios where different organizations hold qualitatively different types of instructional data.

D. Training Configuration

Each client performs local training with the following hyperparameters: 1 local epoch per round, batch size of 2, learning rate of 2×10^{-4} with linear warmup, maximum sequence length of 256 tokens, and FP16 mixed-precision training. The data collator uses causal language modeling (next-token prediction) with automatic label shifting. Token accuracy is computed as the fraction of correctly predicted tokens at non-padded positions.

E. Evaluation Metrics

We track five primary metrics across rounds:

1) **Training loss**: average cross-entropy loss over local training data.

TABLE I
EXPERIMENTAL CONFIGURATIONS

Run	Partition	Clients/Round	Rounds
IID Baseline	IID	4/8	6
Non-IID Baseline	Non-IID	4/8	6
High Client Frac.	Non-IID	6/8	6
More Rounds	Non-IID	4/8	10

2) **Validation loss**: cross-entropy loss on held-out validation data evaluated after aggregation.
3) **Perplexity**: computed as $\text{PPL} = \exp(\mathcal{L}_{\text{val}})$.
4) **Token accuracy**: fraction of correctly predicted next tokens on validation data.
5) **Client validation loss gap**: defined as $\Delta = \max_k \mathcal{L}_k - \min_k \mathcal{L}_k$, serving as a fairness/heterogeneity metric.

Communication cost is measured as the total bytes transmitted per round, computed as $2 \times |\theta_{\text{LoRA}}|$ (accounting for both server-to-client and client-to-server transfers).

IV. EXPERIMENTAL SETUP

We conduct four experiments to systematically evaluate the effects of data distribution, client participation, and communication rounds. Table I summarizes the experimental configurations.

The *IID Baseline* provides a reference point with homogeneous data distribution. The *Non-IID Baseline* introduces data heterogeneity while keeping other factors constant. The *High Client Fraction* experiment increases participation from 50% to 75% of clients per round to examine whether broader participation mitigates heterogeneity effects. The *More Rounds* experiment extends training from 6 to 10 rounds to assess whether additional communication compensates for non-IID degradation.

All experiments were conducted on Google Colab Pro with a single NVIDIA GPU. The Flower simulation engine handles client scheduling, ensuring that each selected client trains sequentially on the same GPU to avoid memory conflicts.

V. RESULTS AND ANALYSIS

A. Convergence Behavior

Figure 1 shows the training and validation loss trajectories for the non-IID baseline configuration over 6 rounds. The training loss exhibits a consistent downward trend from 1.085 in round 1 to 0.866 in round 6, representing a 20.2% reduction. However, the validation loss shows characteristic oscillatory behavior, decreasing initially to 0.943 in round 3 before rising to 1.019 in round 5 and recovering to 0.959 in round 6. This divergence between training and validation trajectories is indicative of the well-known *client drift* phenomenon in federated learning with heterogeneous data [9], where local models overfit to their respective data distributions during local training, causing the aggregated global model to generalize inconsistently.

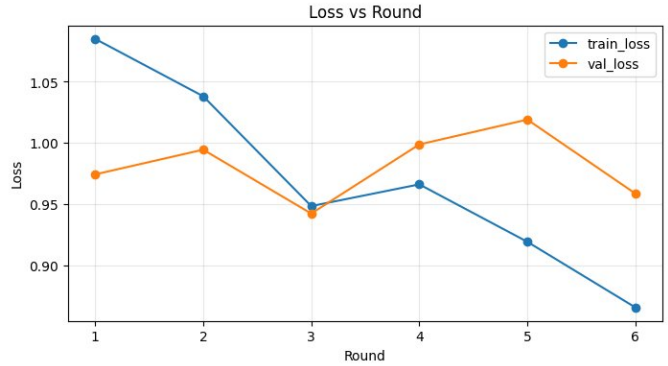

Fig. 1. Training and validation loss vs. communication round for the non-IID baseline configuration. Training loss decreases monotonically while validation loss exhibits oscillatory behavior characteristic of client drift.

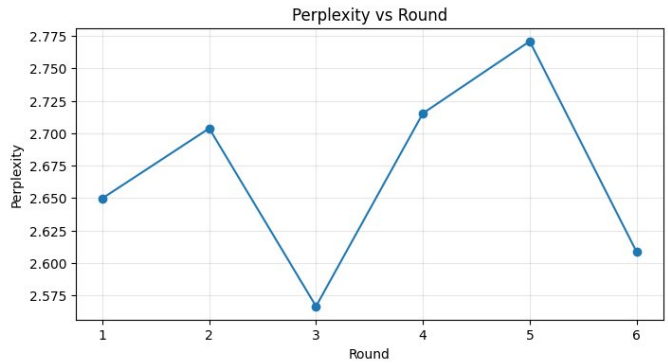

Fig. 2. Validation perplexity vs. communication round. The non-monotonic trajectory reflects the interplay between global aggregation benefits and local overfitting to heterogeneous client distributions.

B. Perplexity Analysis

Figure 2 presents the validation perplexity over communication rounds. As a direct exponential transformation of the validation loss, perplexity mirrors the oscillatory pattern, ranging from a minimum of 2.566 (round 3) to a maximum of 2.771 (round 5). The final-round perplexity of 2.609 represents a modest but meaningful improvement over the first-round value of 2.650. While these absolute perplexity values remain high relative to fully centralized fine-tuning, they demonstrate that the federated procedure is producing directional improvements in language modeling capability even under communication and data constraints.

C. Fairness and Heterogeneity

Figure 3 shows the client validation loss gap, our fairness metric defined as the difference between the highest and lowest client-level validation losses. This metric decreases substantially from 0.787 in round 1 to a minimum of 0.116 in round 5, representing an 85% reduction. The initial high gap reflects the significant variation in data quality and quantity across clients in the non-IID partition (ranging from 20 to 261

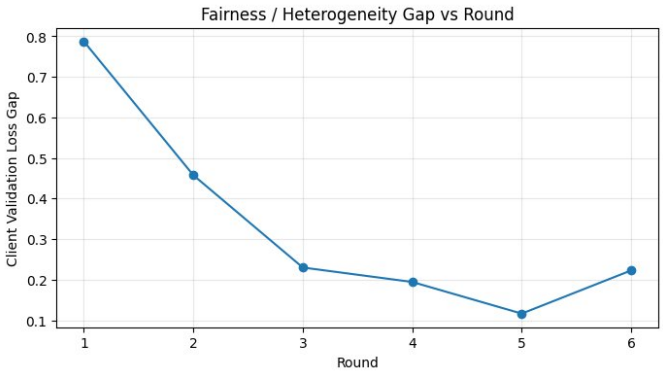

Fig. 3. Client validation loss gap ($\max - \min$ across clients) vs. round. The gap decreases from 0.787 to 0.116, indicating improved fairness through federated aggregation, with a slight rebound in round 6.

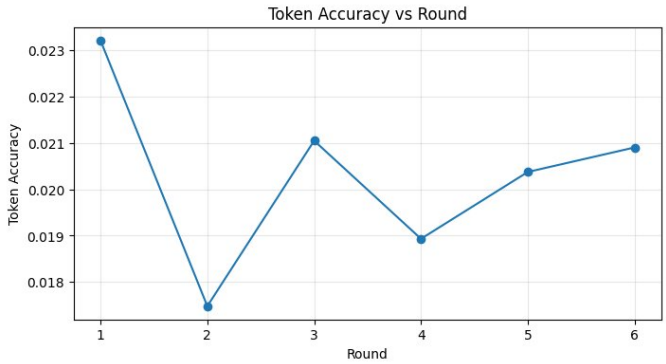

Fig. 4. Token accuracy vs. communication round. Accuracy oscillates in the 1.75–2.32% range, reflecting the difficulty of the instruction-following task under constrained federated training.

training examples). The progressive reduction indicates that FedAvg aggregation effectively distributes knowledge across clients, enabling those with less or different data to benefit from the collective learning of the federation.

The slight increase in round 6 (to 0.223) suggests that fairness convergence is not monotonic and may be sensitive to which specific clients are sampled in each round. This finding motivates future investigation into fairness-aware client selection strategies.

D. Token Accuracy

Figure 4 displays the token-level prediction accuracy across rounds. The accuracy fluctuates between 1.75% and 2.32%, without a clear monotonic improvement trend. The initial accuracy of 2.32% in round 1 slightly decreases in round 2 before recovering, reflecting the adaptation instability common in the early rounds of federated fine-tuning when clients have vastly different data characteristics.

The relatively low absolute accuracy values are expected given the experimental constraints: limited training data (800 examples across 8 clients), a small number of communication rounds (6), a single local epoch per round, and the challenging nature of instruction-following tasks where multiple valid continuations exist. The token accuracy metric measures exact next-token matches, which is a strict criterion that underrepresents the model's actual instruction-following improvement.

E. Communication Efficiency

Figure 5 shows the cumulative communication cost across rounds. Each round transmits approximately 94.4 MB of LoRA adapter parameters (47.2 MB each for download and upload), resulting in a perfectly linear cumulative cost of 566.2 MB over 6 rounds. By comparison, transmitting the full Phi-2 model weights in FP32 would require approximately 10.8 GB per round, or 64.8 GB over 6 rounds. Our approach achieves a **communication reduction factor of approximately 114**$\times$, demonstrating the substantial efficiency gains of LoRA-based federated fine-tuning.

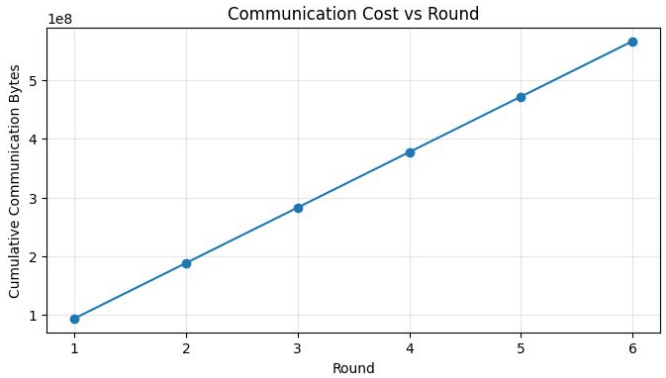

Fig. 5. Cumulative communication cost vs. round. The linear growth at $\sim$94.4 MB per round represents a 114$\times$ reduction compared to transmitting full model weights.

F. Cross-Experiment Comparison

Table II presents the final metrics across all four experimental configurations. Several key observations emerge from this comparison.

Impact of Data Heterogeneity. Comparing the IID and non-IID baselines reveals that data heterogeneity has a pronounced effect on model fairness: the client validation loss gap increases from 0.053 (IID) to 0.292 (non-IID), a 5.5$\times$ amplification. Interestingly, the non-IID setting achieves a lower training loss (0.864 vs. 0.938), suggesting that heterogeneous clients may individually fit their local data more effectively, but this local optimization does not transfer as well to the global model, as evidenced by the higher validation loss (1.020 vs. 0.990).

Effect of Client Participation. Increasing client participation from 50% (4/8) to 75% (6/8) while maintaining the non-IID partition yields improvements across most metrics. The validation loss decreases from 1.020 to 0.993, the fairness gap narrows from 0.292 to 0.213 (a 27% reduction), and token accuracy improves from 1.91% to 2.21%. This confirms

TABLE II
FINAL METRICS ACROSS ALL EXPERIMENTAL CONFIGURATIONS

Metric	IID	Non-IID	High CF	More R.
Train Loss	0.938	0.864	0.888	0.783
Val. Loss	0.990	1.020	0.993	0.999
Perplexity	2.692	2.774	2.698	2.714
Token Acc. (%)	1.83	1.91	2.21	2.49
Loss Gap	0.053	0.292	0.213	0.259
Comm. (MB)	540	540	540	900
Time (min)	19.0	18.0	27.0	30.1

that sampling more clients per round provides better coverage of the heterogeneous data distribution, producing a more representative aggregated model. The improvement comes at the cost of longer wall-clock time (27.0 vs. 18.0 minutes) due to sequential client processing in the simulation.

Effect of Additional Rounds. Extending training from 6 to 10 rounds with the non-IID partition yields the lowest training loss (0.783) and highest token accuracy (2.49%) across all experiments. However, the validation loss (0.999) and perplexity (2.714) do not improve proportionally, suggesting diminishing returns in generalization from additional communication rounds. The total communication cost increases by 67% (from 540 MB to 900 MB), while the fairness gap remains elevated at 0.259.

VI. DISCUSSION

Our results highlight several important considerations for practitioners deploying federated LoRA fine-tuning of LLMs.

Communication-Computation Tradeoff. The 114× communication reduction achieved by transmitting only LoRA adapters (∼45 MB per direction) makes federated fine-tuning of multi-billion parameter models practical even over bandwidth-constrained networks. However, each client still requires sufficient GPU memory to load the quantized base model (∼1.7 GB for 4-bit Phi-2) and perform forward and backward passes. This represents a compute-communication asymmetry where the bottleneck shifts from network bandwidth to local compute resources.

Non-IID Robustness. The 5.5× fairness gap amplification under non-IID conditions underscores the need for heterogeneity-aware aggregation strategies. While increasing client participation partially mitigates this effect, it may not be feasible in large-scale deployments with hundreds or thousands of clients. Alternative strategies such as FedProx [9], SCAFFOLD [15], or personalized federated learning approaches warrant investigation in the LoRA fine-tuning context.

Convergence Limitations. The oscillatory validation loss and modest token accuracy improvements suggest that 6–10 rounds of federated fine-tuning with limited data (∼100 examples per client on average) may be insufficient for meaningful instruction-following capability. Scaling the dataset, increasing local training epochs, or employing more sophisticated learning rate schedules may improve convergence, though these modifications must be balanced against the risk of increased client drift.

Evaluation Methodology. Token accuracy, while commonly used, is a limited metric for evaluating instruction-following quality. Future work should incorporate task-specific evaluations such as ROUGE scores, BLEU scores, or LLM-as-a-judge protocols to better capture the qualitative improvements in generated responses.

VII. CONCLUSION

This paper presented a systematic simulation study of federated fine-tuning of large language models using LoRA adapters within the Flower framework. By transmitting only the 11.8M trainable LoRA parameters (0.42% of the 2.7B-parameter Phi-2 model) and employing 4-bit quantization, we achieved a 114× reduction in per-round communication cost compared to full model transmission.

Our four-experiment comparison revealed that: (1) non-IID data partitioning amplifies the client fairness gap by 5.5× compared to IID settings; (2) increasing client participation from 50% to 75% reduces this gap by 27% while improving token accuracy by 16%; and (3) additional communication rounds yield diminishing returns in generalization despite continued improvement in training loss. These findings provide actionable guidance for system designers seeking to balance communication cost, convergence speed, and model fairness in federated LLM fine-tuning deployments.

Future work will explore adaptive aggregation strategies (FedProx, SCAFFOLD), personalized LoRA adapters where each client maintains a partially local model, larger and more diverse datasets, and comprehensive evaluation using generation quality metrics beyond token accuracy.

REFERENCES

[1] T. Brown *et al.*, "Language models are few-shot learners," in *Proc. NeurIPS*, vol. 33, 2020, pp. 1877–1901.
[2] Q. Li *et al.*, "A survey on federated learning: Systems and vision," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–36, 2021.
[3] B. McMahan *et al.*, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017, pp. 1273–1282.
[4] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.
[5] T. Dettmers *et al.*, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. NeurIPS*, vol. 36, 2023.
[6] D. J. Beutel *et al.*, "Flower: A friendly federated learning framework," *arXiv preprint arXiv:2007.14390*, 2020.
[7] Microsoft Research, "Phi-2: The surprising power of small language models," *Microsoft Research Blog*, Dec. 2023.
[8] P. Taori *et al.*, "Stanford Alpaca: An instruction-following LLaMA model," *GitHub repository*, 2023.
[9] T. Li *et al.*, "Federated optimization in heterogeneous networks," in *Proc. MLSys*, 2020.
[10] J. Konečný *et al.*, "Federated learning: Strategies for improving communication efficiency," *arXiv preprint arXiv:1610.05492*, 2016.
[11] S. Caldas *et al.*, "LEAF: A benchmark for federated settings," *arXiv preprint arXiv:1812.01097*, 2018.
[12] N. Hyeon-Woo *et al.*, "FedPara: Low-rank Hadamard product for communication-efficient federated learning," in *Proc. ICLR*, 2022.
[13] G. Zhang *et al.*, "Towards building the federatedGPT: Federated instruction tuning," in *Proc. ICASSP*, 2024.
[14] J. Zhang *et al.*, "FedPETuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models," in *Proc. ACL Findings*, 2023.

[15] S. P. Karimireddy *et al.*, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. ICML*, 2020.